

Solution experimental zeta — version PC ASUS i7

Architecture IA locale allégée, RAG, MCP, GitHub, calcul zêta et dossiers de travail adaptés à 8 Go RAM / GTX 960 4 Go VRAM / Ubuntu 24.04 LTS

Auteur : hprzeta — Date : 26 mai 2026

Cette version remplace la recommandation “gros modèle local / cluster GPU” par une architecture réaliste pour ton PC : ASUS iCore 7, 8 Go RAM, disque 1 To, swap 16 Go, NVIDIA GTX 960 4 Go VRAM, Ubuntu 24.04 LTS.

Principe important : avec 8 Go RAM et 4 Go VRAM, il faut viser des modèles 1.5B à 3B, éventuellement 4B quantifiés. Les modèles 7B peuvent être testés seulement en mode lent avec faible contexte et offload CPU ; les modèles 30B/35B sont exclus en local sur cette machine.

0. Profil matériel et conséquences techniques

Élément	Configuration déclarée	Conséquence pratique
CPU	ASUS iCore 7	Bon pour compilation, Python/Sage/PARI/FLINT et calculs CPU modérés ; privilégier calculs batch courts.
RAM	8 Go	Limiter Docker, navigateur et IDE ; préférer agents CLI ; modèles 1.5B/3B ; créer swap/zram.
Disque	1 To HDD	Beaucoup d’espace mais I/O lent : organiser cache, modèles, résultats et rapports ; éviter index RAG trop lourds.
Swap	16 Go swap	Utile contre les crashes mémoire ; ne remplace pas la RAM, donc éviter charges constantes lourdes.
GPU	NVIDIA GTX 960 4 Go VRAM	CUDA possible mais ancien ; privilégier Ollama/llama.cpp avec petits modèles quantifiés et offload partiel.
OS	Ubuntu 24.04 LTS	Bonne base scientifique : paquets récents, Python, GCC, Docker, NVIDIA drivers, Sage/PARI/FLINT.

La GTX 960 est une carte CUDA Compute Capability 5.2 selon les informations de détection CUDA associées à ce modèle ; elle reste utilisable pour certains backends, mais les piles CUDA/cuDNN récentes optimisent surtout des architectures plus modernes. Sources : [turn5search139], [turn5search135], [turn5search136].

Ubuntu Desktop recommande au moins 4 Go de RAM et 25 Go d’espace disque ; ta machine dépasse le minimum mais reste limitée pour les agents IA locaux. Sources : [turn5search126].

Sur Ubuntu 24.04, un swapfile peut aider à stabiliser les charges quand la RAM est limitée ; les guides Ubuntu 24.04 expliquent la création d’un fichier swap via fallocation, chmod, mkswap et swapon. Sources : [turn5search127], [turn5search129].

1. IA recommandée adaptée à ce PC

Sur cette configuration, OpenHands reste possible mais ne doit plus être le cœur permanent du système, car Docker + agent + navigateur + modèle local peuvent saturer 8 Go RAM. La meilleure stratégie devient : Aider en CLI pour coder, Continue.dev ou VS Code léger pour relire, Ollama/llama.cpp pour modèles 1.5B/3B, et OpenHands seulement ponctuellement pour tâches courtes en sandbox. OpenHands indique que les LLM locaux peuvent avoir des fonctionnalités limitées et recommande fortement des GPU pour les modèles locaux ; son modèle recommandé Qwen3.6-35B-A3B demande au moins 24 Go VRAM en quantifié, donc il n’est pas adapté à ce PC. Source : [turn5search118].

Usage	Outil recommandé sur ton PC	Pourquoi
Codage agentique léger	Aider + Ollama local	CLI Git-native, peu lourd, facile à rollback.
IDE/relecture	VS Code + Continue.dev	Utilisable avec petit modèle local ou API distante ponctuelle.
Agent autonome	OpenHands ponctuel	À réserver aux tâches courtes ; éviter les grosses sessions Docker.
Modèle local	Qwen2.5-Coder 1.5B ou 3B Q4	Compatible avec 8 Go RAM / 4 Go VRAM ; bon pour code simple.
RAG local	ChromaDB/SQLite + BM25	Simple et léger ; pas de serveur lourd.
Calcul math	Python + mpmath, SageMath, PARI/GP, FLINT si installable	Calculs fiables CPU ; GPU non indispensable pour zêta de base.

2. Modèles locaux réalistes

Les guides Ollama indiquent qu’avec 3-4 Go VRAM, les modèles 3B-4B en quantification Q4 peuvent tourner avec des contextes modérés ; avec 8 Go RAM système, la mémoire réellement disponible pour un modèle peut tomber vers 3-5 Go si l’OS, le navigateur et l’IDE sont ouverts. Sources : [turn5search115], [turn5search116], [turn5search113].

Qwen2.5-Coder existe en tailles 0.5B, 1.5B, 3B, 7B, 14B et 32B ; pour ton PC, les tailles 1.5B et 3B sont prioritaires. Le 7B peut nécessiter environ 5-6 Go en Q4 selon les estimations Ollama, donc il est trop juste avec 4 Go VRAM et seulement 8 Go RAM, sauf test lent avec offload CPU et faible contexte. Sources : [turn5search130], [turn5search131], [turn5search132], [turn5search133].

Priorité	Modèle	Commande Ollama	Usage	Avis hprzeta
1	qwen2.5-coder:1.5b	ollama pull qwen2.5-coder:1.5b	Code, scripts, tests simples	Meilleur premier choix.
2	qwen2.5-coder:3b	ollama pull qwen2.5-coder:3b	Code plus robuste, PR petites	Choix principal si stable.
3	llama3.2:3b	ollama pull llama3.2:3b	Planification, explication, résumé	Bon second cerveau généraliste.
4	phi3.5:mini ou phi4-mini si disponible	ollama pull phi3.5	Raisonnement compact	À tester selon vitesse.
5	qwen2.5-coder:7b	ollama pull qwen2.5-coder:7b	Code meilleur mais lourd	Option expérimentale lente, faible contexte.
Non	Qwen3.6-35B-A3B / 30B / 32B	—	Agentic coding fort	À exclure localement : VRAM/RAM insuffisantes.

3. Paramètres d’exécution modèle

Paramètres recommandés pour éviter les blocages : fermer navigateur lourd, limiter contexte, utiliser quantification Q4, un seul agent LLM à la fois, et préférer les tâches courtes. Le disque HDD rend les chargements de modèles et index RAG plus lents ; garder les modèles et caches dans un dossier clair.

```
# Profils Ollama conseillés
Profil rapide : qwen2.5-coder:1.5b, contexte 4096, température 0.0
Profil principal : qwen2.5-coder:3b, contexte 4096 à 8192, température 0.0
Profil résumé : llama3.2:3b, contexte 4096, température 0.1
Profil expérimental : qwen2.5-coder:7b, contexte 2048 à 4096, fermer toutes les autres applications

# Règle de travail
1 modèle chargé à la fois ; 1 agent actif ; pas de Docker OpenHands + navigateur + gros IDE simultanément.
```

4. Agents spécialisés adaptés à machine limitée

Au lieu de faire tourner plusieurs agents simultanément, on garde une architecture “multi-agents séquentielle”. Chaque agent est un rôle/prompt exécuté à tour de rôle dans Aider/Ollama ou dans un script LangGraph très léger. Cela donne les bénéfices de spécialisation sans saturer RAM/VRAM.

Agent	Mode sur ce PC	Limite
planner	LLM 3B, génère issues et tâches	Pas de web lourd ; RAG local limité.
coder	Aider + qwen2.5-coder 1.5B/3B	Petites PR seulement.
tester	pytest, scripts Python sans LLM si possible	Automatisé, peu de RAM.
zeta_computation	Python/mpmath/Sage/PARI CPU	Calculs modestes, blocs courts.
verifier	Comparaison LMFDB/valeurs stockées + tolérances	Pas de preuve massive.
rag_librarian	ChromaDB/SQLite local	Index restreint aux sources utiles.
report_writer	Markdown + PDF via Python	Rapports compressés, pas de gros médias.
github_devops	Git CLI + GitHub web/gh	MCP GitHub optionnel si léger.
OpenHands	Session courte, tâche unique	Stopper si swap élevé.

5. Architecture de dossier de travail sur 1 To

Je recommande de séparer clairement : dépôt Git, données de référence, résultats, modèles IA, caches, RAG et sauvegardes. Comme ton disque est un HDD 1 To, il faut éviter que les caches Docker/Ollama/RAG envahissent le système. Utilise un dossier racine unique, par exemple /home/\$USER/hprzeta-lab.

```
/home/$USER/hprzeta-lab/
00_README_LOCAL.md
01_env/                                # environnements Python, scripts setup, notes drivers
  requirements-minimal.txt
  environment.yml
  setup_ubuntu24.sh
  hardware_profile.md
02_models/                             # modèles locaux GGUF/Ollama exportés si besoin
  ollama_notes.md
  model_tests/
03_rag/                                # base documentaire RAG légère
  sources/
    report_hprzeta.pdf
    solution_experimental_zeta_pc_asus.pdf
  papers/
  docs_tools/
  indexes/
    chroma_small/
    bm25/
  metadata.sqlite
04_projects/
  hprzeta/                             # dépôt GitHub principal cloné ici
    README.md
    pyproject.toml
    src/hprzeta/
    tests/
    notebooks/
    docs/
```

```

data/
lean/
hpc/
.agents/
.github/
05_data_reference/          # petites tables de référence, pas les énormes bases
  lmfdb_first_zeros.csv
  odlyzko_sample.csv
  checksums.txt
06_results/                 # sorties expérimentales générées
  experiments/
  plots/
  logs/
07_reports/                 # copies PDF/Markdown finales
  daily/
  releases/
08_backups/                 # archives périodiques hors Git
  repo_snapshots/
  rag_snapshots/
09_tmp/                     # temporaire nettoyable
10_docker/                  # docker compose OpenHands optionnel
  openhands-light/

```

Organisation interne du dépôt GitHub hprzeta

```

hprzeta/
  README.md
  CITATION.cff
  LICENSE
  pyproject.toml
  Makefile
  src/hprzeta/
    zeta/                # Hardy Z, theta, zeta wrappers
    zeros/               # localisation, raffinage, comparaison
    verification/        # tolérances, checks, Turing method simplifiée
    primes/              # primalité, pi(x), fonctions arithmétiques
    rag/                 # wrappers RAG légers
    reporting/           # génération markdown/pdf/json
  tests/
    unit/
    reference/
    integration/
  notebooks/
    001_first_zeros.ipynb
    002_hardy_z_validation.ipynb
  docs/
    methodology/
    reports/
    sources_manifest.md
  data/
    reference/           # petits fichiers versionnés
    results/             # petits résultats ; gros résultats hors Git
    manifests/
  lean/HPRZeta/          # expérimental, seulement si Lean stable sur machine
  .agents/
  prompts/
  skills/
  workflows/
  .github/workflows/
  tests.yml
  lint.yml
  docs.yml

```

6. Installation minimale recommandée

Avec 8 Go RAM, éviter les installations lourdes au départ. Installer d'abord l'outillage minimal, puis Sage/Lean/OpenHands seulement si la machine reste stable. SageMath, PARI/GP et FLINT restent importants pour théorie des nombres ; FLINT fournit notamment primalité, factorisation, FFT, fonctions spéciales et arithmétique à boules. Sources : [turn1search13], [turn1search55], [turn1search43].

```

# Base Ubuntu 24.04
sudo apt update
sudo apt install -y git curl build-essential cmake pkg-config python3 python3-venv python3-pip pari-gp libgmp-
dev libmpfr-dev libmpc-dev libflint-dev ffmpeg pandoc texlive-latex-base

# Python projet
cd ~/hprzeta-lab/04_projects/hprzeta
python3 -m venv .venv
source .venv/bin/activate
pip install -U pip wheel
pip install numpy scipy mpmath sympy pandas matplotlib pytest ruff mypy jupyterlab chromadb rank-bm25 reportlab
pypdf

# Ollama
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5-coder:1.5b
ollama pull qwen2.5-coder:3b
ollama pull llama3.2:3b

# Aider
pip install aider-chat

```

Swap et stabilité

Si ton swap 16 Go n'est pas déjà actif, créer un swapfile. Les guides Ubuntu 24.04 montrent les commandes fallocation, chmod, mkswap, swapon et ajout dans /etc/fstab. Sources : [turn5search127], [turn5search129].

```

# Vérifier swap
free -h
swapon --show

# Exemple si besoin : créer 16 Go de swapfile

```

```
sudo fallocate -l 16G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

# Réduire usage agressif du swap
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-hprzeta.conf
sudo sysctl --system
```

7. Choix RAG léger

Le RAG doit rester petit : indexer le rapport hprzeta, le PDF solution, quelques articles et la documentation locale, pas des gigaoctets. Utiliser ChromaDB local + BM25 ; limiter chunks à 500-900 tokens et garder les métadonnées. Les guides RAG recommandent chunking, embeddings, base vectorielle, recherche hybride et citations. Sources : [turn3search94], [turn3search95], [turn3search97].

RAG local conseillé :

- Embeddings : petit modèle local si disponible, sinon sentence-transformers léger.
- Vector DB : ChromaDB dans ~/hprzeta-lab/03_rag/indexes/chroma_small/
- Lexical : rank-bm25 dans ~/hprzeta-lab/03_rag/indexes/bm25/
- Sources limitées : 20 à 50 documents maximum au départ.
- Citations obligatoires : fichier, page/section, hash, date d'indexation.

8. MCP et connecteurs adaptés

Sur cette machine, MCP doit rester minimal. Filesystem MCP et GitHub MCP suffisent au départ. Le protocole MCP standardise ressources, prompts et tools pour connecter LLM et outils ; le serveur filesystem offre lecture/écriture/search avec contrôle d'accès, et GitHub MCP peut gérer repo, issues, PR et logs CI. Sources : [turn3search107], [turn3search105], [turn3search106], [turn3search108].

Connecteur	Statut sur ton PC	Conseil
Filesystem MCP	Oui	Limiter à ~/hprzeta-lab/04_projects/hprzeta et docs utiles.
GitHub MCP	Oui/optionnel	Read-only par défaut ; écriture uniquement validation humaine.
Shell MCP	Très contrôlé	Autoriser pytest, git status, python scripts ; bloquer rm -rf, sudo.
Docker/OpenHands	Optionnel	Lancer ponctuellement ; fermer Ollama gros modèle avant.
Slurm/HPC	Non local	Préparer scripts, mais exécution distante future.
Lean MCP	Optionnel	Uniquement si Lean installé et build léger.

9. Workflow quotidien réaliste

Le workflow doit être séquentiel pour économiser la RAM : planifier, coder, tester, vérifier, rapporter, puis commit. Éviter de laisser tourner simultanément OpenHands, VS Code, navigateur lourd, Jupyter et Ollama 7B.

Routine quotidienne hprzeta PC ASUS :

1. Démarrer : free -h ; swapon --show ; nvidia-smi si pilote disponible.
2. Activer venv : source ~/hprzeta-lab/04_projects/hprzeta/.venv/bin/activate
3. Lancer un modèle léger : ollama run qwen2.5-coder:1.5b ou 3b
4. Planifier une seule issue.
5. Coder avec Aider sur une petite branche feature/*.
6. Lancer pytest -q.
7. Comparer les résultats à data/reference/.
8. Générer rapport Markdown/PDF/JSON.
9. git diff ; git commit ; git push ; PR.
10. Fermer le modèle : ollama stop <model> si besoin.

10. Calcul zêta adapté à cette machine

Ton PC est suffisant pour reproduire les premiers zéros, tester Hardy Z(t), comparer à LMFDB, générer des plots et valider de petits intervalles. Il n'est pas adapté à une campagne massive de milliards de zéros. Pour les grandes hauteurs, préparer du code propre puis exécuter plus tard sur cluster distant. LMFDB fournit une référence de zéros et Platt illustre les exigences HPC pour une vérification massive. Sources : [turn1search1], [turn1search5], [turn1search6], [turn1search22].

Niveau	Objectif	Faisable sur ce PC ?
Niveau 1	10 à 1000 premiers zéros, comparaison tables	Oui
Niveau 2	Hardy Z(t), Riemann-Siegel simple, plots	Oui
Niveau 3	Intervalles avec haute précision et logs	Oui, petits blocs
Niveau 4	Méthode de Turing complète et grands T	Partiel / prototype
Niveau 5	Millions/milliards de zéros	Non local ; cluster requis

11. Prompt maître modifié pour ton PC

Tu es hprzeta-orchestrator-local, directeur scientifique et technique d'un projet expérimental zêta sur PC ASUS limité.

Matériel : Intel i7, 8 Go RAM, 16 Go swap, 1 To HDD, NVIDIA GTX 960 4 Go VRAM, Ubuntu 24.04 LTS.

Contraintes matérielles :

- Utiliser prioritairement qwen2.5-coder:1.5b ou 3b.
- Ne pas lancer plusieurs agents LLM en parallèle.
- Ne pas utiliser de modèle 30B/35B localement.
- Réserver OpenHands aux tâches courtes ; préférer Aider CLI.
- Limiter contexte à 4096 ou 8192 tokens.
- Sauvegarder tous les rapports et logs.
- Ne jamais prétendre prouver RH.

- Priorités :
1. Installer environnement minimal.
 2. Reproduire premiers zéros.
 3. Créer tests de comparaison.
 4. Ajouter RAG local léger.
 5. Générer rapports PDF.
 6. Préparer architecture future cluster, sans exécuter de charge massive localement.

12. Prompts spécialisés modifiés

Agent coder local

Tu es hprzeta-coder-local.
Tu travailles sur une machine limitée : 8 Go RAM, 4 Go VRAM, HDD.
Tu dois produire de petites modifications, une fonction à la fois.
Avant toute modification : git status.
Après modification : pytest ciblé, puis rapport court.
Tu n'utilises pas de gros modèle et tu n'ouvres pas de gros fichiers inutilement.

Agent RAG local

Tu es hprzeta-rag-local.
Tu indexes uniquement les documents nécessaires.
Tu limites la taille des chunks et tu conserves les métadonnées : fichier, page, section, hash, date.
Tu refuses de répondre sans source si la question demande une affirmation technique.

Agent calcul local

Tu es hprzeta-zeta-local.
Tu réalises uniquement des calculs adaptés à la machine : premiers zéros, petits intervalles, validation par référence.
Tu ne lances pas de campagne massive.
Tu journalises : paramètres, temps CPU, RAM, précision, référence utilisée.

13. GitHub et sauvegarde

Le dépôt GitHub ne doit contenir que le code, les petits fichiers de référence, les notebooks légers et les rapports. Les gros résultats, modèles Ollama, caches Chroma et archives restent hors Git, dans ~/hprzeta-lab/06_results, 02_models et 08_backups. Utiliser .gitignore strict.

```
.gitignore recommandé :
.venv/
__pycache__/
.ipynb_checkpoints/
*.gguf
*.bin
*.safetensors
.ollama/
rag/indexes/chroma*/
*.sqlite-wal
*.sqlite-shm
data/results/large/
logs/
*.tmp
*.bak
.DS_Store
```

14. CI/CD allégée

Sur GitHub Actions, garder la CI légère : ruff, pytest sur petits tests, génération d'un rapport minimal. Les tests lourds doivent être marqués slow et non exécutés par défaut.

CI minimale :

- ruff check src tests
- pytest -q tests/unit tests/reference
- python scripts/generate_report.py --quick

Marqueurs pytest :

- @pytest.mark.slow # non lancé par défaut
- @pytest.mark.local_only # dépend de machine
- @pytest.mark.reference # compare aux tables connues

15. Plan de démarrage 30 jours adapté

Période	Objectif réaliste	Livrables
Semaine 1	Préparer dossier hprzeta-lab, swap, venv, Ollama 1.5B/3B, dépôt Git.	hardware_profile.md, README_LOCAL.md, premier commit.
Semaine 2	Implémenter premiers zéros avec mpmath/Python et référence CSV.	notebook 001, tests reference, rapport PDF.
Semaine 3	Installer PARI/GP/FLINT si stable, ajouter seconde méthode de comparaison.	module verification, rapport écarts.
Semaine 4	Ajouter RAG local léger, prompts agents, workflow Aider + GitHub PR.	.agents, index RAG, templates issues/PR.

16. Ce qu'il faut éviter sur cette machine

- Ne pas utiliser Qwen3.6-35B-A3B, Qwen 30B/32B, OpenHands LM 32B ou tout modèle supérieur à 7B en local.
- Ne pas lancer OpenHands + Docker + Ollama 7B + VS Code + navigateur simultanément.
- Ne pas indexer des gigaoctets de PDF dans ChromaDB sur HDD.
- Ne pas lancer une campagne de calcul massive de zéros sur le PC local.
- Ne pas stocker modèles, caches RAG et gros résultats dans le dépôt Git.
- Ne pas considérer le swap comme de la vraie RAM : il évite le crash mais ralentit fortement.

17. Recommandation finale adaptée

La combinaison finale pour ton PC est : Ubuntu 24.04 LTS + swap 16 Go + Ollama/llama.cpp + qwen2.5-coder:1.5b/3b + Aider CLI + VS Code/Continue.dev léger + Python/mpmath/Sage/PARI/FLINT + ChromaDB/BM25 local + GitHub + rapports PDF. OpenHands devient optionnel et ponctuel, pas le moteur permanent. Cette architecture maximise la stabilité tout en gardant la possibilité de pousser vers un futur cluster distant.

18. Sources citées

- GTX 960 CUDA capability et CUDA — [turn5search139 / turn5search135]
- Support CUDA/cuDNN Ubuntu 24.04 et architectures modernes — [turn5search136]
- Ubuntu minimum requirements — [turn5search126]
- Swap Ubuntu 24.04 — [turn5search127 / turn5search129]
- OpenHands local LLM requirements — [turn5search118]
- Ollama petit matériel / 4 Go VRAM / 8 Go RAM — [turn5search115 / turn5search116 / turn5search113]
- Qwen2.5-Coder tailles et VRAM — [turn5search130 / turn5search131 / turn5search132 / turn5search133]
- RAG bonnes pratiques — [turn3search94 / turn3search95 / turn3search97]
- MCP filesystem et GitHub — [turn3search105 / turn3search106 / turn3search108]
- LMFDB et zéros de $\zeta(s)$ — [turn1search1 / turn1search5 / turn1search6]
- Platt et calcul massif de zéros — [turn1search22]
- FLINT, PARI/GP, SageMath — [turn1search13 / turn1search55 / turn1search43]